

**SYSTÈMES NUMÉRIQUES
POUR L'HUMAIN**
ÉCOLE UNIVERSITAIRE DE RECHERCHE

EUR DS4H - MASTER 1 INFORMATIQUE PARCOURS INTELLIGENCE ARTIFICIELLE

Projet : Operations research

Hugo VIANA

1 Introduction

Les problèmes de flot dans les graphes constituent une famille fondamentale de problèmes d'optimisation combinatoire et de recherche opérationnelle. Ils apparaissent dans de nombreux domaines tels que les réseaux de transport, la logistique, les télécommunications, l'allocation de ressources ou encore les systèmes industriels. Le problème du flot maximum consiste à déterminer la quantité maximale pouvant être acheminée d'une source vers un puits dans un réseau capacitaire, tandis que les variantes de flot à coût minimum cherchent à optimiser simultanément la quantité transportée et le coût associé au transport.

L'objectif de ce projet est de proposer une implémentation modulaire en langage C de plusieurs algorithmes classiques de théorie des flots. Le solveur développé permet le calcul de flots maximaux, de flots à coût minimum ainsi que la détection de cycles négatifs dans un graphe résiduel. Une attention particulière a été portée à la conception logicielle afin de produire une architecture extensible, maintenable et adaptée à l'expérimentation algorithmique.

Le projet repose également sur un système de visualisation dynamique basé sur Graphviz permettant de représenter l'évolution du réseau résiduel au cours des différentes étapes d'exécution des algorithmes. Cette approche facilite à la fois la compréhension théorique des méthodes implémentées et le débogage des structures de données manipulées.

Les algorithmes implémentés sont Ford-Fulkerson avec recherche de chemin augmentant par parcours en profondeur, deux variantes de flot à coût minimum reposant respectivement sur Bellman-Ford et Dijkstra avec potentiels, ainsi qu'un algorithme de détection de cycles négatifs fondé sur Bellman-Ford multi-source.

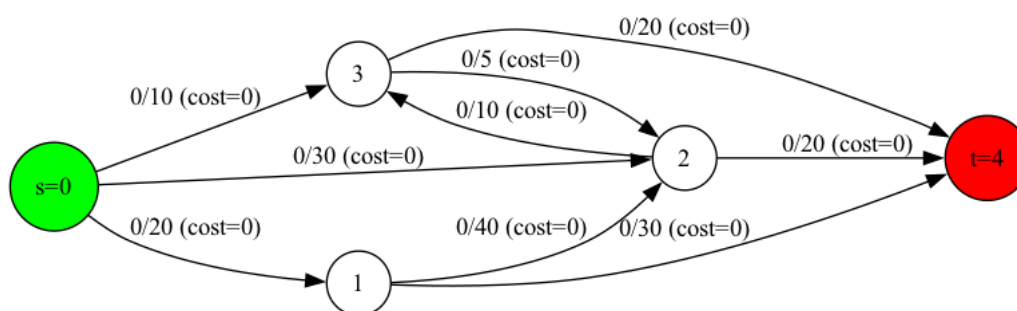


Figure 1: Exemple de graphe de flot.

2 Modélisation du graphe et choix de conception

La qualité d'une implémentation d'algorithmes de flot dépend fortement des structures de données utilisées pour représenter le graphe et le réseau résiduel. Dans ce projet, plusieurs choix de conception ont été effectués afin de privilégier la simplicité des opérations de mise à jour, l'efficacité mémoire et l'extensibilité du solveur.

2.1 Représentation du graphe

Le graphe est représenté à l'aide de listes d'adjacence. Chaque sommet possède une liste chaînée contenant l'ensemble de ses arcs sortants. Cette structure est définie par les types

Node et Graph.

```
typedef struct Node {
    Arc* arcs;
} Node;

typedef struct Graph {
    int num_nodes;
    int s, t;
    Node* nodes;
} Graph;
```

Le choix des listes d'adjacence plutôt qu'une matrice d'adjacence est motivé principalement par des considérations de complexité mémoire. Une matrice nécessite un espace mémoire en $O(n^2)$, ce qui devient rapidement prohibitif pour les graphes creux. À l'inverse, une représentation par listes d'adjacence nécessite seulement $O(n + m)$, où n représente le nombre de sommets et m le nombre d'arcs.

Ce choix est particulièrement adapté aux algorithmes de flot, qui parcourent essentiellement les arcs sortants de chaque sommet lors des recherches de chemins augmentants ou des relaxations de distances. Les listes d'adjacence permettent ainsi de limiter les parcours inutiles et de réduire la consommation mémoire globale.

2.2 Structure des arcs

Les arcs sont représentés par la structure suivante :

```
typedef struct Arc {
    int node_to;
    int min_capacity;
    int capacity;
    int cost;
    int flow;
    struct Arc* residual;
    struct Arc* next;
} Arc;
```

Chaque arc contient l'identifiant du sommet destination, la capacité maximale, le coût unitaire ainsi que le flot actuellement transporté. Le champ `residual` constitue un pointeur vers l'arc résiduel associé, tandis que `next` permet le chaînage dans la liste d'adjacence.

Le champ `flow` stocke explicitement le flot courant traversant l'arc. La capacité résiduelle d'un arc est alors donnée par :

$$c_f(u, v) = c(u, v) - f(u, v)$$

où $c(u, v)$ désigne la capacité de l'arc et $f(u, v)$ le flot actuellement envoyé.

2.3 Gestion du graphe résiduel

Le graphe résiduel constitue l'élément central des algorithmes de flot. Lors de l'ajout d'un arc dans le graphe, un arc inverse résiduel est automatiquement créé. Les deux arcs sont reliés mutuellement à l'aide du champ `residual`.

```
Arc* forward_arc = allocate_arc(...);
Arc* backward_arc = allocate_arc(...);

forward_arc->residual = backward_arc;
backward_arc->residual = forward_arc;
```

Cette conception permet d'accéder immédiatement à l'arc inverse lors des augmentations de flot. Une mise à jour du flot peut alors être effectuée en temps constant :

```
void push_flow(Arc* a, int delta) {
    a->flow += delta;
    a->residual->flow -= delta;
}
```

Cette relation garantit la conservation antisymétrique du flot :

$$f(u, v) = -f(v, u)$$

L'utilisation de pointeurs directs vers les arcs résiduels simplifie fortement l'implémentation des algorithmes de flot et évite des recherches coûteuses dans les listes d'adjacence.

2.4 Gestion des bornes inférieures

La structure des arcs contient également un champ `min_capacity`, destiné à représenter une borne inférieure sur le flot :

$$l(e) \leq f(e) \leq c(e)$$

Bien que cette fonctionnalité ne soit pas exploitée dans les algorithmes actuellement implémentés, sa présence permet d'anticiper une extension future du solveur vers les problèmes de circulation avec bornes inférieures. Ces modèles apparaissent fréquemment dans les applications industrielles de transport et de planification.

L'intégration anticipée de ce champ illustre une volonté de conception extensible permettant d'ajouter ultérieurement des contraintes plus générales sans modifier profondément les structures internes du graphe.

3 Algorithme de Ford-Fulkerson

L'algorithme de Ford-Fulkerson constitue l'un des algorithmes fondamentaux de théorie des flots. Son principe repose sur l'augmentation itérative d'un flot réalisable à l'aide de chemins augmentants dans le graphe résiduel. Tant qu'un chemin reliant la source au puits possède une capacité résiduelle strictement positive, il est possible d'augmenter le flot global.

Le théorème fondamental du flot maximum établit que la valeur du flot maximal est égale à la capacité d'une coupe minimale :

$$\max(f) = \min(C)$$

où f désigne un flot réalisable et C une coupe séparant la source du puits.

3.1 Recherche de chemin augmentant

Dans cette implémentation, la recherche de chemin augmentant repose sur un parcours en profondeur itératif. Le choix d'une version itérative plutôt que récursive permet de mieux contrôler l'utilisation mémoire et d'éviter les risques de dépassement de pile sur des graphes de grande taille.

La fonction principale de recherche est définie par :

```
bool dfs(Graph* graph, Arc** parent, bool* visited)
```

L'algorithme maintient explicitement une pile contenant les sommets à explorer. Lorsqu'un arc possédant une capacité résiduelle strictement positive est rencontré, le sommet destination est marqué comme visité et l'arc correspondant est mémorisé dans le tableau des parents.

Le tableau `parent` permet ensuite de reconstruire le chemin augmentant trouvé entre la source et le puits.

3.2 Calcul du goulot d'étranglement

Une fois le chemin augmentant déterminé, il est nécessaire d'identifier la quantité maximale de flot pouvant être ajoutée sur ce chemin. Cette valeur correspond au minimum des capacités résiduelles rencontrées :

$$\Delta = \min_{e \in P} c_f(e)$$

où P représente le chemin augmentant.

Cette opération est réalisée par la fonction :

```
int bottleneck(Graph* graph, Arc** parent)
```

Le parcours est effectué en remontant le tableau des parents depuis le puits jusqu'à la source.

3.3 Augmentation du flot

L'augmentation du flot consiste à ajouter la valeur Δ sur chacun des arcs du chemin augmentant. Simultanément, le flot inverse est soustrait sur les arcs résiduels correspondants.

Cette étape est implémentée dans la fonction :

```
void augment(Graph* graph, Arc** parent, int delta)
```

Grâce aux pointeurs directs vers les arcs résiduels, chaque mise à jour est réalisée en temps constant.

3.4 Calcul de la coupe minimale

Une fois qu'aucun chemin augmentant n'existe dans le graphe résiduel, le flot obtenu est maximal. Une recherche supplémentaire dans le graphe résiduel permet alors d'identifier la coupe minimale associée.

Les sommets encore accessibles depuis la source dans le graphe résiduel forment un ensemble S , tandis que les autres sommets forment l'ensemble complémentaire T . Les arcs saturés reliant S à T constituent alors la coupe minimale.

Cette propriété illustre directement le théorème max-flow/min-cut.

3.5 Complexité

La complexité de Ford-Fulkerson dépend fortement de la méthode utilisée pour rechercher les chemins augmentants. Avec une recherche par DFS, la complexité est pseudo-polynomiale :

$$O(E \cdot |f^*|)$$

où E désigne le nombre d'arcs et $|f^*|$ la valeur du flot maximal.

Cette dépendance à la valeur du flot peut conduire à des performances médiocres sur certains graphes. L'ordre de sélection des chemins augmentants influence directement le nombre total d'itérations nécessaires à la convergence.

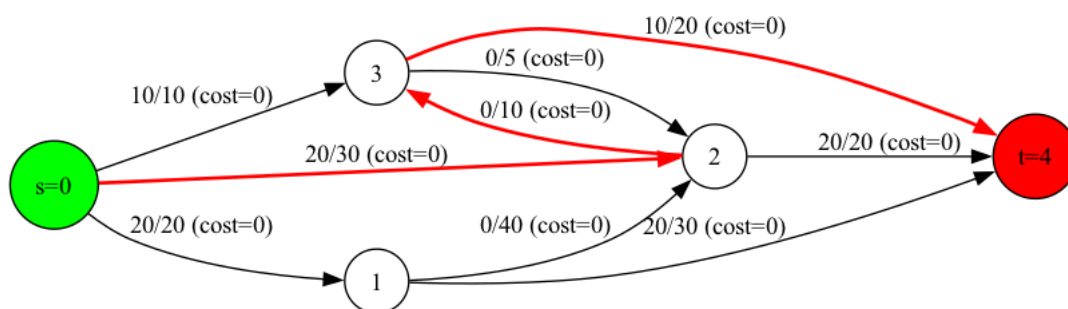


Figure 2: Chemin augmentant.

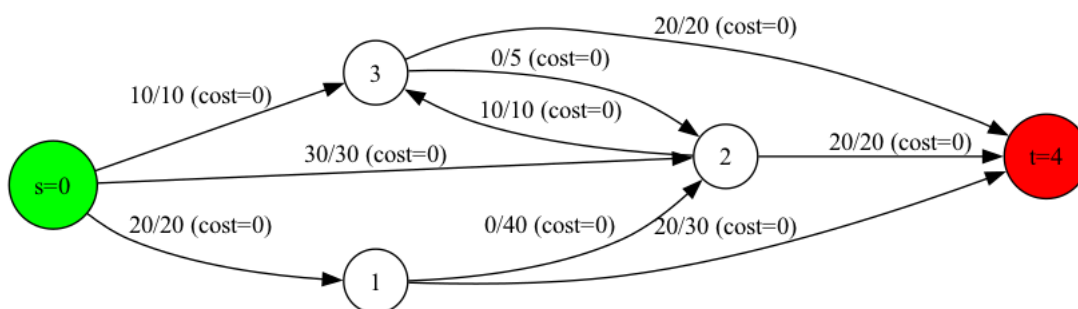


Figure 3: Flot augmenté.

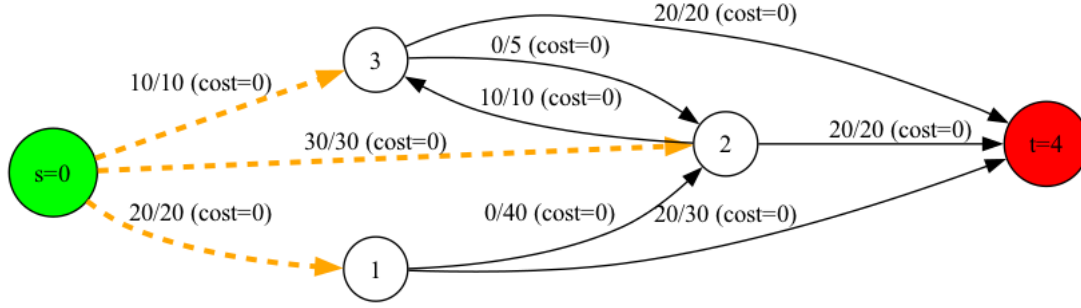


Figure 4: Coupe min.

4 Algorithmes de flot à coût minimum

Le problème du flot à coût minimum généralise le problème du flot maximum en introduisant un coût associé au transport d'une unité de flot sur chaque arc du graphe. L'objectif consiste alors à minimiser le coût total du flot tout en respectant les contraintes de capacité et de conservation.

Le coût total d'un flot est défini par :

$$\sum_{(u,v) \in E} c(u,v)f(u,v)$$

où $c(u,v)$ représente le coût unitaire de l'arc et $f(u,v)$ le flot transporté.

Deux approches ont été implémentées dans ce projet : une version reposant sur Bellman-Ford et une version utilisant Dijkstra avec potentiels.

4.1 Bellman-Ford

L'algorithme de Bellman-Ford est utilisé afin de trouver un plus court chemin dans le graphe résiduel même en présence de coûts négatifs. Cette propriété est essentielle dans les problèmes de flot à coût minimum puisque les arcs résiduels inverses possèdent naturellement des coûts négatifs.

La méthode repose sur des relaxations successives des arcs du graphe. Pour chaque sommet v , une estimation de la distance minimale depuis la source est maintenue. Lorsqu'un arc permet d'améliorer cette estimation, la distance ainsi que le parent du sommet sont mis à jour.

Le processus est répété au plus $n - 1$ fois, où n représente le nombre de sommets du graphe.

La complexité d'une exécution de Bellman-Ford est :

$$O(VE)$$

où V désigne le nombre de sommets et E le nombre d'arcs.

Dans le cadre du flot à coût minimum, cette opération est répétée à chaque augmentation de flot, ce qui conduit à une complexité totale :

$$O(FVE)$$

où F représente la valeur totale du flot envoyé.

4.2 Dijkstra avec potentiels

Bien que Dijkstra soit plus rapide que Bellman-Ford, il ne peut pas être utilisé directement en présence de coûts négatifs. Afin de contourner cette limitation, l'algorithme implémenté utilise une technique de repondération inspirée des potentiels de Johnson.

Chaque sommet u possède un potentiel $\pi(u)$. Le coût réduit d'un arc devient alors :

$$c'(u, v) = c(u, v) + \pi(u) - \pi(v)$$

Cette transformation conserve l'optimalité des chemins tout en garantissant des coûts réduits non négatifs. Dijkstra peut alors être appliqué efficacement sur le graphe résiduel repondéré.

Après chaque exécution, les potentiels sont mis à jour à partir des distances calculées :

```
potentials[i] += dist[i];
```

Cette propriété garantit la conservation de coûts réduits positifs lors des itérations suivantes.

L'implémentation actuelle utilise une sélection linéaire du sommet minimal. La complexité d'une itération est donc :

$$O(V^2)$$

Une version utilisant un tas binaire permettrait d'obtenir une complexité asymptotique plus favorable :

$$O((V + E) \log V)$$

4.3 Comparaison des approches

L'approche Bellman-Ford présente l'avantage d'être simple et de gérer naturellement les coûts négatifs. En revanche, sa complexité devient rapidement importante sur des graphes de grande taille.

L'approche utilisant Dijkstra avec potentiels est généralement plus efficace en pratique. Le coût supplémentaire lié au maintien des potentiels est compensé par une recherche de plus court chemin beaucoup plus rapide.

Le choix entre les deux méthodes dépend principalement de la taille du graphe et des performances recherchées.

5 Détection de cycles négatifs

La détection de cycles négatifs joue un rôle important dans les problèmes de flot à coût minimum. La présence d'un cycle de coût total négatif dans le graphe résiduel signifie qu'il est possible de diminuer le coût global du flot en faisant circuler du flot le long de ce cycle.

L'algorithme implémenté repose sur Bellman-Ford multi-source. Contrairement à une version classique démarrant depuis une source unique, toutes les distances sont initialisées

à zéro. Cette approche permet de détecter également les cycles négatifs non atteignables depuis la source du réseau.

À chaque relaxation, la condition suivante est vérifiée :

$$dist[u] + c(u, v) < dist[v]$$

Si une relaxation reste possible après n itérations, alors un cycle négatif existe dans le graphe.

L'algorithme mémorise le dernier sommet mis à jour afin de reconstruire ensuite le cycle correspondant. Pour garantir que le sommet appartient effectivement au cycle, plusieurs remontées successives dans le tableau des parents sont effectuées avant l'extraction finale.

Le cycle est ensuite reconstruit en parcourant les parents jusqu'au retour au sommet de départ.

La complexité de la détection est :

$$O(VE)$$

tandis que l'extraction du cycle nécessite au plus :

$$O(V)$$

Cette fonctionnalité constitue une base importante pour des extensions futures vers des algorithmes plus avancés de circulation à coût minimum.

6 Architecture logicielle

Une attention particulière a été portée à l'organisation du projet afin de produire une architecture modulaire, lisible et facilement extensible. Les différentes responsabilités ont été séparées dans plusieurs modules indépendants.

Le module `graph.c` regroupe l'ensemble des opérations liées à la création et à la manipulation des graphes. Les algorithmes de flot maximum sont implémentés dans `ford_fulkerson.c`, tandis que les variantes de flot à coût minimum sont regroupées dans `min_cost.c`. La détection de cycles négatifs est isolée dans `negative_cycle.c`. Enfin, les fonctionnalités de visualisation sont centralisées dans `viz.c`.

Cette séparation favorise la maintenabilité du code et limite les dépendances entre les différentes parties du projet.

Le solveur repose également sur une forme d'abstraction algorithmique via l'utilisation de pointeurs de fonctions. La fonction Ford-Fulkerson reçoit en paramètre une stratégie de recherche de chemin augmentant :

```
int ford_fulkerson(Graph* graph,
                  find_path_fn find_path,
                  const char* output_dir)
```

Cette approche permet de remplacer facilement la stratégie de parcours utilisée sans modifier le cœur de l'algorithme. Une implémentation future d'Edmonds-Karp pourrait par exemple être ajoutée simplement en introduisant une recherche BFS compatible avec cette interface.

Le programme propose également une interface en ligne de commande permettant de sélectionner dynamiquement l'algorithme utilisé :

```
./flowSolver --algo ff  
./flowSolver --algo mcf-bf  
./flowSolver --algo mcf-dijkstra
```

Cette approche facilite les expérimentations et améliore la reproductibilité des tests.

Enfin, un système de visualisation basé sur Graphviz permet de générer automatiquement une représentation du graphe à chaque étape importante des algorithmes. Les chemins augmentants sont affichés en rouge tandis que les coupes minimales sont mises en évidence à l'aide d'arcs orange en pointillés.

Cette visualisation constitue un outil particulièrement utile pour comprendre le comportement dynamique des algorithmes de flot.

7 Validation expérimentale

La validation du projet repose exclusivement sur les tests unitaires implémentés dans le dépôt. Ces tests ont été conçus afin de vérifier les propriétés fondamentales des structures de données et des algorithmes implémentés.

Les tests utilisent la bibliothèque standard `assert` du langage C. Chaque fonction critique du solveur possède ainsi plusieurs cas de validation ciblés.

Les tests du module graphe vérifient notamment la création correcte des structures de données, l'ajout des arcs ainsi que la cohérence des liens résiduels entre arcs directs et arcs inverses.

Concernant Ford-Fulkerson, plusieurs graphes de test ont été utilisés afin de valider la recherche de chemins augmentants, le calcul du goulot d'étranglement, l'augmentation du flot ainsi que le calcul du flot maximal. Les cas étudiés incluent un chemin unique, des chemins parallèles, un goulot d'étranglement ainsi qu'un graphe sans chemin entre la source et le puits.

Une vérification supplémentaire confirme également l'égalité entre la valeur du flot maximal et la capacité de la coupe minimale extraite après convergence.

Les tests des algorithmes de flot à coût minimum valident le calcul des plus courts chemins, la gestion des coûts négatifs sans cycle négatif ainsi que le choix optimal entre plusieurs chemins concurrents.

Enfin, les tests de détection de cycles négatifs couvrent plusieurs situations distinctes : absence de cycle négatif, présence d'un cycle négatif atteignable, présence d'un cycle négatif isolé ainsi qu'un cycle strictement positif. Les tests vérifient également la cohérence de l'extraction du cycle et la négativité effective du coût total obtenu.

Ces tests permettent de valider le comportement fonctionnel des principales composantes du solveur. En revanche, aucune campagne de benchmark de performance ni analyse systématique sur des graphes de grande taille n'a été réalisée dans le cadre de ce projet.

8 Conclusion et perspectives

Ce projet a permis de développer un solveur de flot modulaire en langage C intégrant plusieurs algorithmes fondamentaux de recherche opérationnelle. L'implémentation couvre le calcul de flot maximal, le calcul de flot à coût minimum ainsi que la détection de cycles négatifs dans les graphes résiduels.

L'architecture retenue privilégie la modularité et l'extensibilité. L'utilisation de structures explicites pour les arcs résiduels ainsi que la séparation claire des différents modules facilitent l'évolution future du solveur.

Le système de visualisation basé sur Graphviz constitue également un apport important du projet. Il permet de suivre précisément les différentes étapes des algorithmes et offre un support pédagogique particulièrement utile pour l'analyse des réseaux de flot.

Plusieurs perspectives d'amélioration peuvent être envisagées. Une première extension naturelle consisterait à implémenter Edmonds-Karp afin d'obtenir une complexité théorique bornée en $O(VE^2)$. L'intégration de l'algorithme de Dinic permettrait également d'améliorer significativement les performances sur des graphes de grande taille.

Concernant le flot à coût minimum, l'ajout de structures de données plus efficaces telles que des tas binaires permettrait d'optimiser l'implémentation actuelle de Dijkstra. Une implémentation de SPFA pourrait également être étudiée comme alternative pratique à Bellman-Ford.

Enfin, le champ `min_capacity` déjà présent dans les structures ouvre la voie à l'implémentation future de problèmes de circulation avec bornes inférieures, ainsi qu'à des problèmes d'affectation ou de transport plus généraux.